

```
.....
private:
    void flushAVar(TracedVar *v);
};
```

Step 3: To modify the file *agent.cc* in the common folder, write the following code:

```
Agent::Agent(packet_t pkttype) :
    size_(0), type_(pkttype), frametype_(0),
    channel_(0), traceName_(NULL),
    oldValueList_(NULL), app_(0), et_(0)
{
}
.....
Agent::initpkt(Packet* p) const
{
    hdr_cmn* ch = hdr_cmn::access(p);
    ch->uid() = uidcnt_++;
    ch->ptype() = type_;
    ch->size() = size_;
    ch->timestamp() = Scheduler::instance().clock();
    ch->iface() = UNKN_IFACE.value(); // from packet.h (agent
is local)
    ch->direction() = hdr_cmn::NONE;

    ch->error() = 0; // pkt not corrupt to start with */
// add the following line
    ch->frametype_ = frametype_;
.....
```

Step 4: Copy the *myevalvid* folder (which contains *myevalvid.cc*, *myudp.cc*, *myudp.h*, *myevalvid_sink.cc* and *myevalvid_sink.h*) into ns-2.35; for example, *ns-allinone-2.35/ns-2.35/myevalvid*.

Step 5: To modify *ns-allinone-2.35/ns-2.35/tcl/lib/ns-default.tcl*, add the following two lines...

```
Agent/myUDP set packetSize_ 1000
Tracefile set debug_ 0
```

Step 6: To modify *ns-allinone-2.35/ns-2.35/Makefile.in*, put *myevalvid_sink.h* into ns2.35; for example, *ns-allinone-2.35/ns-2.35/myevalvid.o* in the *OBJ_CC* list.

Step 7: Recompile ns-2 as follows:

```
./configure ; make clean ; make
```

Formulating video traces from raw video and simulation

Assume that EvalVid and ns-2 are installed on Ubuntu. Similar steps will work on other Linux distros too.

Download a raw yuv video sequence (you can use known yuv videos from <http://trace.eas.asu.edu/yuv/>). Here is an

1	I	15876	16	0.002
2	P	1223	2	0.049
3	P	1116	2	0.084
4	P	937	1	0.103
5	P	1163	2	0.140
6	P	1168	2	0.178
7	P	1079	2	0.215
8	P	1234	2	0.234
9	P	1290	2	0.272
10	P	1388	2	0.310
11	P	1230	2	0.347
12	P	1383	2	0.384
13	P	1727	2	0.403
14	P	2016	2	0.440
15	P	2387	3	0.478

Figure 4: Snap of source video trace

example where *bus_cif.yuv* is used as raw video.

1. Encode the raw yuv file to *m4v* as follows:

```
$ffmpeg -s cif -r 30 -b 64000 -bt 3200 -g 30 -i bus_cif.yuv
-vcodec mpeg4 bus_cif.m4v
```

2. Convert the *m4v* file to *mp4*:

```
$MP4Box -hint -mtu 1024 -fps 30 -add bus_cif.m4v bus_cif2.mp4
```

As an option, you can create a reference yuv file. This video can be used for encoding loss estimation:

```
$ffmpeg -i bus_cif2.mp4 bus_cif_ref.yuv
```

3. Send an *mp4* file per RTP/UDP to a specified destination host. The output of the *mp4trace* will be needed later, so it should be redirected to a file. The *Source_Video_Trace* file is the video trace file which has the information about each frame of the video and will be used by ns-2 as a traffic source.

```
$mp4trace -f -s 224.1.2.3 12346 bus_cif2.mp4 >
Source_Video_Trace
```

(A section of *Source_Video_Trace* is shown in Figure 4.)

4. Create a network topology representing the network to be simulated in *Evalvid.tcl* and run the simulation as follows:

```
$ns Evalvid.tcl
```

5. After simulation, ns-2 will create two files, *Send_time_file* and *Recv_time_file* (the filename as used in the *tcl* file), which record the sending time and received time of each packet, respectively.
6. To reconstruct the transmitted video as it is seen by the receiver, the video and trace files are processed by *etmp4* (Evaluate Traces of MP4-file transmission):

To be continue on page... 89